

Ontwerp van een geautomatiseerd systeem voor security monitoring en incident response in SaaS omgevingen binnen de DevOps-infrastructuur van Devinity.

Sem Demoen.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. S. Samyn

Co-promotor: Dhr. T. Neels

Instelling: Devinity BV.

Academiejaar: 2025–2026

Tweede examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Samenvatting

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Inhoudsopgave

Lijst van figuren	vii
Lijst van tabellen	viii
Lijst van codefragmenten	ix
1 Inleiding	1
1.1 Probleemstelling	2
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	4
1.4 Opzet van deze bachelorproef	4
2 Stand van zaken	6
3 Methodologie	8
3.1 Fase 1: Probleemanalyse	8
3.2 Fase 2: Architectuurontwerp.	8
3.3 Fase 3: Proof-of-concept ontwikkeling	9
3.4 Fase 4: Evaluatie.	9
4 Probleemanalyse	10
5 Architectuurontwerp	11
6 Proof-of-concept ontwikkeling	12
7 Evaluatie	13
8 Conclusie	14

A	Onderzoeksvoorstel	16
A.1	Inleiding	16
A.2	Literatuurstudie	18
A.3	Methodologie	20
A.4	Verwacht resultaat, conclusie	21
	Bibliografie	23

Lijst van figuren

3.1	Overzicht van de methodologie in vier fasen.	9
-----	--	---

Lijst van tabellen

Lijst van codefragmenten

1

Inleiding

Deze bachelorproef onderzoekt hoe een geautomatiseerd systeem voor security monitoring en incident response kan worden ontworpen om securitydata binnen SaaS-omgevingen effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur, en bouwt hier verder op aan de hand van een uitgewerkt proof-of-concept. Het onderzoek vertrekt van een casus bij Devinity, een SaaS-gericht bedrijf dat momenteel meerdere tools/platformen gebruikt voor security monitoring en incidentdetectie. Dit leidt tot problemen zoals vertraagde detectie, onvolledige rapporten en inconsistentie in het afhandelen van incidenten.

De doelgroep bestaat uit het DevOps- en securityteam binnen Devinity, die baat hebben bij een gecentraliseerd en geautomatiseerd systeem dat de operationele efficiëntie verhoogt en risico's beter beheersbaar maakt. Binnen de scope van dit onderzoek wordt gefocust op één representatief SaaS-platform en één DevOps-pipeline om de haalbaarheid en diepgang van het proof-of-concept te verzekeren.

Hoewel het onderzoek zich inhoudelijk situeert binnen het domein van Security Information and Event Management (SIEM), wordt geen volledige enterprise-SIEM-oplossing ontwikkeld. In plaats daarvan worden kernprincipes van SIEM toegepast binnen een afgebakende SaaS- en DevOps-context. Concreet ligt de focus op logaggregatie, normalisatie, eventcorrelatie en rapportering.

Deze focus sluit aan bij bestaande literatuur, waarin wordt benadrukt dat security in DevOps-omgevingen geïntegreerd moet worden in de ontwikkel- en deployment-processen om snelle detectie en effectieve respons mogelijk te maken (**GruhnKohntopp2021**). Het ontwikkelen van een geautomatiseerde en geïntegreerde monitoringoplossing sluit dan ook rechtstreeks aan bij deze aanbeveling.

1.1. Probleemstelling

De toenemende adoptie van Software-as-a-Service (SaaS) en DevOps-praktijken heeft geleid tot sterk geautomatiseerde en dynamische omgevingen waarin beveiligingsmonitoring een cruciale rol speelt. Binnen zulke omgevingen worden dan ook continu grote hoeveelheden securitydata gegenereerd die op een efficiënte en gestructureerde manier verwerkt moeten worden, wat niet altijd zo simpel blijkt voor bedrijven. Recente rapporten tonen aan dat onder andere data-beveiliging een belangrijke uitdaging vormt voor organisaties (Tuyishime e.a., [2023](#)).

Volgens (Jakku, [2025](#)) vormen monitoring en logging de basis voor effectieve DevOps-praktijken. Monitoring biedt inzicht in de gezondheid van de systemen, prestatie-metingen en potentiële knelpunten, terwijl logging gedetailleerde gegevens vastlegt over gebeurtenissen, fouten en activiteiten.

Devinity vormt hierin geen uitzondering en krijgt dan ook dagelijks te maken met een concrete uitdaging binnen het vlak van security monitoring en incident response in de eigen SaaS- en DevOps-omgeving. Momenteel worden veel logs, events en alerts bekeken op verschillende afzonderlijke tools en platformen. Hierdoor vergroot de kans dat een incident te laat wordt gedetecteerd, rapportages inconsistent of onvolledig zijn, en dat analyses manueel moeten uitgevoerd worden.

Voor de DevOps engineers en functional testers binnen Devinity betekent dit dat ze geen praktisch en gecentraliseerd overzicht hebben over de beveiligingsstatus van de infrastructuur. Daarnaast krijgt het managementteam geen duidelijke en uniforme rapportages, waardoor risicomanagement heel wat minder doelmatig verloopt.

Het ontbreken van een geautomatiseerde en gecentraliseerde oplossing heeft een directe impact op de analytische efficiëntie, de snelheid van incidentdetectie en de kwaliteit van de besluitvorming binnen Devinity.

Dit toont aan dat er binnen Devinity nood is aan een oplossing die logs, events en alerts uit de gebruikte platformen en omgevingen centraal verzamelt, deze analyseert en automatisch rapporteert. Op deze manier kan het bedrijf incidenten sneller detecteren, onnodige waarschuwingen beperken en een duidelijker overzicht krijgen van de beveiligingsstatus binnen het bedrijf.

1.2. Onderzoeksvraag

De onderzoeksvraag luidt als volgt: Hoe kan een geautomatiseerd systeem voor security monitoring en incident response worden ontworpen om securitydata binnen de SaaS-omgeving van Devinity effectief te centraliseren, analyseren en rap-

porteren in een DevOps-infrastructuur?

De **probleemstelling** wordt verder uitgewerkt aan de hand van volgende deelonderzoeksvragen:

1. Welke uitdagingen ervaren SaaS-bedrijven zoals Devinity bij het gebruik van meerdere tools voor security monitoring en incidentdetectie?
2. Hoe worden incidenten momenteel gedetecteerd en gerapporteerd, en welke tekortkomingen bestaan er in snelheid, correlatie en consistentie van rapporten?
3. Welke securitydata (logs, events, alerts) worden beschikbaar gesteld door bestaande systemen en hoe worden deze momenteel gecentraliseerd en geanalyseerd?
4. Welke metrics en KPIs worden nu gebruikt om de effectiviteit van security monitoring en incident response te meten, en welke zijn onvoldoende of inconsistent?
5. Hoe beïnvloedt de huidige aanpak van monitoring en rapportering de operationele efficiëntie en het risicomanagement van SaaSbedrijven?

De uitwerking van de **oplossing** gebeurt aan de hand van onderstaande deelonderzoeksvragen:

1. Welke architectuur en technologieën zijn het meest geschikt om een geautomatiseerd securitymonitoringsysteem te ontwikkelen dat past binnen een DevOps-omgeving,
2. Hoe kan eenvoudige anomaly-detectie op SaaS login-auditlogs, als aanvulling op rule-based detectieregels, bijdragen aan het identificeren van ongebruikelijke authenticatiepogingen?
3. Welke data-integratieprocessen (APIs, agents, pipelines) zijn het meest efficiënt om logs en securitydata uit het geselecteerde SaaS-platform en de DevOps-pipeline te verzamelen en te normaliseren?
4. Hoe kan automatische rapportgeneratie worden geïmplementeerd zodat rapporten zowel technische details als managementinzichten bevatten?
5. Welke prestatie-indicatoren (KPIs) kunnen gebruikt worden om het succes van het systeem te meten, zoals detectiesnelheid, foutpositieven of responstijd?
6. Hoe kan de beveiliging en betrouwbaarheid van het geautomatiseerde systeem zelf worden gegarandeerd binnen de bestaande DevOps-pipeline?

1.3. Onderzoeksdoelstelling

Het doel van deze bachelorproef is het ontwikkelen van een proof-of-concept met het vermogen om securitydata uit een geselecteerd SaaS-platform en DevOps-omgeving automatisch te verzamelen en verwerken. Om ervoor te zorgen dat alle relevante gegevens centraal binnen het systeem terechtkomen kunnen verschillende technieken gebruikt worden, waaronder APIs, webhooks en logforwarders. Na het verzamelen van de data is het de bedoeling dat deze testimplementatie de gegevens structureert en normaliseert zodat ze op een efficiënte manier geanalyseerd kunnen worden.

Na het verwerken van deze gegevens zal er een automatisch rapport opgesteld worden dat de belanghebbenden uit het bedrijf voorziet van bruikbare informatie over de beveiligingsstatus om zo incidenten te voorkomen. Deze informatie kan eventueel ook KPIs bevatten om de effectiviteit van monitoring en incident response te meten.

Het systeem moet het dan ook mogelijk maken om sneller inzicht te krijgen in mogelijke incidenten, trends te herkennen in security-events en het overzicht van de infrastructuur te verbeteren, zodat het bedrijf geïnformeerde beslissingen kan nemen en de incidentrespons kan optimaliseren.

Concreet is het eindresultaat een werkend prototype (proof-of-concept) dat de voordelen van centralisatie, snelle detectie en consistente rapportage aantoont voor SaaS- en DevOps gerichte bedrijven zoals Devinity.

Om dit doel te realiseren, wordt het onderzoek uitgevoerd in vier fasen, verder beschreven in Hoofdstuk 3:

1. Probleemanalyse
2. Architectuurontwerp
3. Proof-of-concept ontwikkeling
4. Evaluatie

1.4. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt een overzicht gegeven van de stand van zaken binnen het onderzoeksdomein, op basis van een literatuurstudie.

In Hoofdstuk 3 wordt de methodologie toegelicht en worden de gebruikte onder-

zoekstechnieken besproken om een antwoord te kunnen formuleren op de onderzoeksvragen.

In Hoofdstuk 4 wordt de eerste fase van het onderzoek behandeld: de probleemanalyse. Hierin wordt de huidige situatie onderzocht, bestaande monitoringtools en processen in kaart gebracht, en worden relevante logs, incidenten en KPI's geanalyseerd.

In Hoofdstuk 5 wordt het architectuurontwerp gepresenteerd, gebaseerd op de bevindingen uit de probleemanalyse. Er wordt ingegaan op de geselecteerde technologieën, de datastromen en de kerncomponenten van het centrale security-systeem.

In Hoofdstuk 6 wordt de implementatie van het proof-of-concept beschreven. Dit hoofdstuk behandelt de opzet van het prototype, de ontwikkelde kernfuncties zoals logverzameling en detectiemechanismen, en de uitgevoerde tests met simulatie- en testdata.

In Hoofdstuk 7 worden de resultaten van het proof-of-concept geanalyseerd. De prestaties en effectiviteit van het systeem worden beoordeeld en er worden conclusies getrokken en aanbevelingen gedaan voor optimalisatie en toekomstig gebruik.

In Hoofdstuk 8, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen. Daarbij wordt ook een aanzet gegeven voor toekomstig onderzoek binnen dit domein.

2

Stand van zaken

De laatste jaren is er een enorme groei in de nood voor complexe netwerkinfrastructuren en cloudgebaseerde diensten binnen bedrijven in de IT-sector. Dit heeft geleid tot een aanzienlijke toename in het aantal beveiligingstools en monitoring-oplossingen (Persistence Market Research, [2025](#)). Ondanks deze evolutie blijkt het voor heel wat organisaties nog steeds moeilijk om een doordachte en doeltreffende architectuur voor security monitoring te ontwerpen en te implementeren (Obregon, [2015](#)). De gevolgen van deze eerder moeizame implementatie uiten zich onder meer in dataverlies door onvoldoende zichtbaarheid op security-events en netwerkverkeer, verhoogde kost in nieuwe technologieën en nood aan extra personeel om het overweldigend aantal alerts te verwerken (Obregon, [2015](#)).

Het decentraliseren van de virtuele architectuur binnen een onderneming biedt voordelen, zoals verhoogde schaalbaarheid, flexibiliteit en wendbaarheid. Netwerksegmentatie speelt dan ook zeker een belangrijke rol binnen een informatiebeveiligingsstrategie: door het netwerk op te delen in verschillende zones neemt de kans dat een succesvolle aanval zich verspreidt aanzienlijk af, terwijl de zichtbaarheid van het netwerk tegelijkertijd wordt vergroot, *je kan namelijk niet beschermen wat je niet kan zien* (Obregon, [2015](#)). Echter brengt het opsplitsen van een dergelijk netwerk ook een aanzienlijke problemen met zich mee, met name de verhoogde complexiteit bij het beveiligen van de systemen (Billawa e.a., [2022](#)). Het monitoren van elk afzonderlijk systeem binnen een netwerk is doorgaans ook niet haalbaar vanuit een financieel perspectief en kan bovendien leiden tot een verkeerd gevoel van beveiligingszekerheid binnen de organisatie (**Obegron_2015**). Daarom is het van cruciaal belang om op een doordachte manier om te gaan met deze recente fragmentatie.

"prevention is ideal, but detection is a must" (Dr. Eric Cole, geciteerd in Obregon, 2015) (**Obregon2015**). Binnen de informatiebeveiliging bestaat er een fundamentele trade-off tussen **preventie** en **detectie** bij het implementeren van beveiligingsmaatregelen. Preventie richt zich op het blokkeren van aanvallen vóórdat ze schade veroorzaken, terwijl detectie focust op het identificeren en analyseren van verdachte activiteiten zodra ze plaatsvinden. Volgens de Guide to Intrusion Detection and Prevention Systems (IDPS) kunnen Intrusion Prevention Systems (IPS) automatisch kwaadaardig verkeer blokkeren, bijvoorbeeld door sessies te beëindigen, pakketten te droppen of firewallregels dynamisch aan te passen (Scarfone & Mell, 2007). Intrusion Detection Systems (IDS) daarentegen genereren voornamelijk waarschuwingen, waarna verdere analyse of manuele interventie nodig is. Deze detectiesystemen verminderen het risico op fout-positieven, aangezien ze geen directe impact hebben op het netwerkverkeer. Aan de andere kant is er dan wel meer monitoring en expertise nodig om deze data correct te interpreteren en tijdig op te volgen.

De NIST-richtlijnen benadrukken dat een effectieve beveiligingsstrategie niet enkel steunt op preventie of detectie afzonderlijk, maar op een gelaagde benadering waarbij beide complementair worden ingezet. Preventieve mechanismen beperken het aanvalsoppervlak en blokkeren gekende dreigingen, terwijl detectieve systemen zorgen voor zichtbaarheid, analyse van incidenten en het identificeren van nieuwe of geavanceerde aanvallen (Scarfone & Mell, 2007). Naarmate architecturen meer en meer gedecentraliseerd worden, neemt de nood toe aan verhoogde zichtbaarheid en het centraal analyseren van beveiligingsevents, terwijl tegelijkertijd moet worden vermeden dat preventieve controles de processen binnen de organisatie verstoren.

3

Methodologie

Het onderzoek wordt uitgevoerd in vier fasen, die samen een gestructureerd en iteratief plan van aanpak vormen. Elke fase draagt bij aan het ontwikkelen en evalueren van een proof-of-concept voor security monitoring en incident response binnen een SaaS- en DevOps-context.

3.1. Fase 1: Probleemanalyse

In de eerste fase wordt in samenwerking met Devinity de huidige situatie onderzocht. Huidige monitoringtools en problemen worden in kaart gebracht. Ook worden bestaande logs, incidenten en KPI's geanalyseerd. Het doel is een duidelijk inzicht te krijgen in de huidige problemen en uitdagingen, zodat het ontwerp van het systeem gericht en haalbaar kan zijn. Het resultaat van deze fase is een analyseverslag van de huidige situatie en probleemstelling, gebaseerd op observaties, verschillende interviews en een diepgaande loganalyse.

3.2. Fase 2: Architectuurontwerp

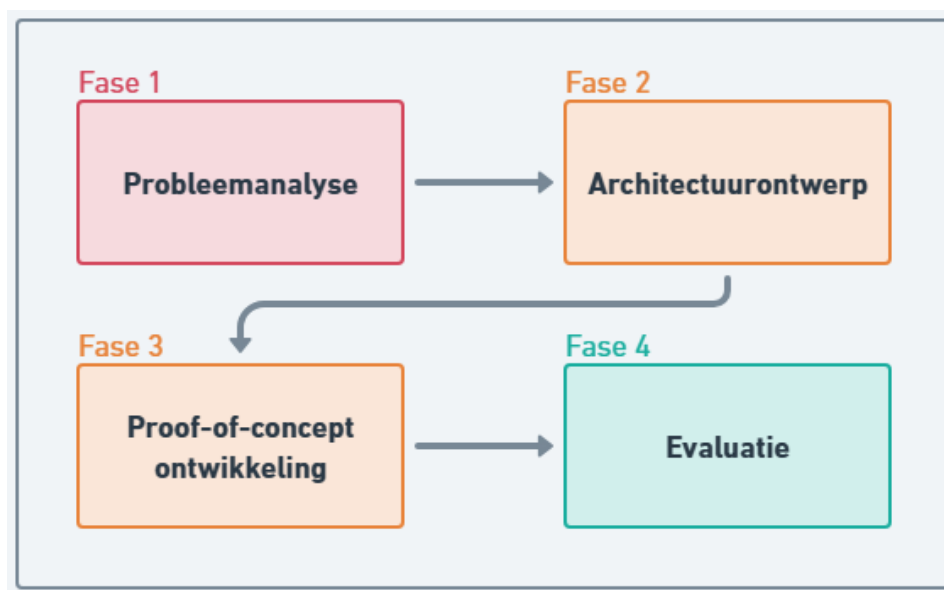
Op basis van de probleemanalyse wordt een architectuur ontworpen voor de centrale verzameling en verwerking van de securitydata. Verschillende technologieën worden geselecteerd en de datastromen worden gedefinieerd via data pipelines zoals API's, webhooks en logforwarders. Deze fase bouwt voort op de kernprincipes van SIEM-systemen en NIST-richtlijnen. Het resultaat is een systeemontwerp van de architectuur en datastromen.

3.3. Fase 3: Proof-of-concept ontwikkeling

In deze fase wordt het prototype geïmplementeerd binnen de geselecteerde SaaS- en DevOps-omgeving. Kernfuncties zoals logverzameling, regelgebaseerde detectie, eenvoudige anomaly-detectie en automatische rapportage worden ontwikkeld. Testen worden uitgevoerd met gesimuleerde data en ongevoelige logdata uit testomgevingen. Deze testdata bestaat uit authenticatielogs, auditlogs, API-events en doelbewust gegenereerde security-events zoals mislukte logins of configuratiefouten om realistische scenario's te kunnen simuleren. Het resultaat is een werkend proof-of-concept systeem dat de centralisatie, analyse en rapportage van security-data aantoont.

3.4. Fase 4: Evaluatie

Tot slot worden de prestaties van het PoC geëvalueerd. Deze evaluatie maakt het mogelijk om de effectiviteit van het systeem te beoordelen en aanbevelingen voor optimalisatie op te stellen. Het resultaat van deze fase is een evaluatierapport met conclusies en aanbevelingen voor het bedrijf.



Figuur 3.1: Schematisch overzicht van de vierfasen-methodologie van dit onderzoek.

De aanpak bouwt voort op de NIST-aanbevelingen voor intrusion detection en prevention systemen (Scarfone & Mell, 2007), waarin het vastleggen en analyseren van security events centraal staat voor effectieve detectie en respons. Tegelijkertijd baseert de architectuurkeuze zich op kerncomponenten van SIEM-systemen, zoals gecentraliseerde logverzameling, normalisatie en event-monitoring (González-Granadillo e.a., 2021).

4

Probleemanalyse

5

Architectuurontwerp

6

Proof-of-concept ontwikkeling

7

Evaluatie

8

Conclusie

Curabitur nunc magna, posuere eget, venenatis eu, vehicula ac, velit. Aenean ornare, massa a accumsan pulvinar, quam lorem laoreet purus, eu sodales magna risus molestie lorem. Nunc erat velit, hendrerit quis, malesuada ut, aliquam vitae, wisi. Sed posuere. Suspendisse ipsum arcu, scelerisque nec, aliquam eu, molestie tincidunt, justo. Phasellus iaculis. Sed posuere lorem non ipsum. Pellentesque dapibus. Suspendisse quam libero, laoreet a, tincidunt eget, consequat at, est. Nullam ut lectus non enim consequat facilisis. Mauris leo. Quisque pede ligula, auctor vel, pellentesque vel, posuere id, turpis. Cras ipsum sem, cursus et, facilisis ut, tempus euismod, quam. Suspendisse tristique dolor eu orci. Mauris mattis. Aenean semper. Vivamus tortor magna, facilisis id, varius mattis, hendrerit in, justo. Integer purus.

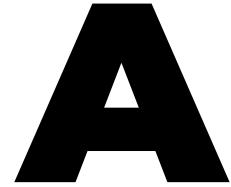
Vivamus adipiscing. Curabitur imperdiet tempus turpis. Vivamus sapien dolor, congue venenatis, euismod eget, porta rhoncus, magna. Proin condimentum pretium enim. Fusce fringilla, libero et venenatis facilisis, eros enim cursus arcu, vitae facilisis odio augue vitae orci. Aliquam varius nibh ut odio. Sed condimentum condimentum nunc. Pellentesque eget massa. Pellentesque quis mauris. Donec ut ligula ac pede pulvinar lobortis. Pellentesque euismod. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent elit. Ut laoreet ornare est. Phasellus gravida vulputate nulla. Donec sit amet arcu ut sem tempor malesuada. Praesent hendrerit augue in urna. Proin enim ante, ornare vel, consequat ut, blandit in, justo. Donec felis elit, dignissim sed, sagittis ut, ullamcorper a, nulla. Aenean pharetra vulputate odio.

Quisque enim. Proin velit neque, tristique eu, eleifend eget, vestibulum nec, lacus. Vivamus odio. Duis odio urna, vehicula in, elementum aliquam, aliquet laoreet, tel-

lus. Sed velit. Sed vel mi ac elit aliquet interdum. Etiam sapien neque, convallis et, aliquet vel, auctor non, arcu. Aliquam suscipit aliquam lectus. Proin tincidunt magna sed wisi. Integer blandit lacus ut lorem. Sed luctus justo sed enim.

Morbi malesuada hendrerit dui. Nunc mauris leo, dapibus sit amet, vestibulum et, commodo id, est. Pellentesque purus. Pellentesque tristique, nunc ac pulvinar adipiscing, justo eros consequat lectus, sit amet posuere lectus neque vel augue. Cras consectetur libero ac eros. Ut eget massa. Fusce sit amet enim eleifend sem dictum auctor. In eget risus luctus wisi convallis pulvinar. Vivamus sapien risus, tempor in, viverra in, aliquet pellentesque, eros. Aliquam euismod libero a sem.

Nunc velit augue, scelerisque dignissim, lobortis et, aliquam in, risus. In eu eros. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Curabitur vulputate elit viverra augue. Mauris fringilla, tortor sit amet malesuada mollis, sapien mi dapibus odio, ac imperdiet ligula enim eget nisl. Quisque vitae pede a pede aliquet suscipit. Phasellus tellus pede, viverra vestibulum, gravida id, laoreet in, justo. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Integer commodo luctus lectus. Mauris justo. Duis varius eros. Sed quam. Cras lacus eros, rutrum eget, varius quis, convallis iaculis, velit. Mauris imperdiet, metus at tristique venenatis, purus neque pellentesque mauris, a ultrices elit lacus nec tortor. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent malesuada. Nam lacus lectus, auctor sit amet, malesuada vel, elementum eget, metus. Duis neque pede, facilisis eget, egestas elementum, nonummy id, neque.



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Inleiding

Deze bachelorproef onderzoekt hoe een geautomatiseerd systeem voor security monitoring en incident response kan worden ontworpen om securitydata binnen SaaS-omgevingen effectief te centraliseren, analyseren en rapporteren in een DevOps-infrastructuur. Het onderzoek vertrekt vanuit een casus bij Devinity.eu, een SaaS-bedrijf dat momenteel meerdere tools gebruikt voor security monitoring en incidentdetectie. Dit leidt tot problemen zoals vertraagde detectie, onvolledige rapporten en inconsistentie in incidentafhandeling. De doelgroep bestaat uit DevOps- en securityteams binnen SaaS-bedrijven, die baat hebben bij een gecentraliseerd en geautomatiseerd systeem dat de operationele efficiëntie verhoogt en risico's beter beheersbaar maakt.

Binnen de scope van dit onderzoek wordt gefocust op één representatief SaaS-platform en één DevOps-pipeline om de haalbaarheid en diepgang van het proof-of-concept te waarborgen. Bovendien benadrukken Gruhn & Köhntopp (2021) dat security in DevOps-omgevingen een geïntegreerde aanpak vereist, waarbij monitoring en incident response nauw verweven zijn met de ontwikkeling- en deploymentprocessen, om snelle detectie en respons op beveiligingsincidenten te kunnen garanderen (Gruhn & Köhntopp, [2021](#)).

Centrale onderzoeksvraag: *Hoe kan een geautomatiseerd systeem voor security*

monitoring en incident response worden ontworpen om securitydata binnen een SaaS-platform effectief te centraliseren, analyseren en rapporteren binnen een DevOps-pipeline?

Deelonderzoeksvragen

Probleemgericht:

1. Welke uitdagingen ervaren SaaS-bedrijven zoals Devinity bij het gebruik van meerdere tools voor security monitoring en incidentdetectie?
2. Hoe worden incidenten momenteel gedetecteerd en gerapporteerd, en welke tekortkomingen bestaan er in snelheid, correlatie en consistentie van rapporten?
3. Welke securitydata (logs, events, alerts) worden beschikbaar gesteld door bestaande systemen en hoe worden deze momenteel gecentraliseerd en geanalyseerd?
4. Welke metrics en KPIs worden nu gebruikt om de effectiviteit van security monitoring en incident response te meten, en welke zijn onvoldoende of inconsistent?
5. Hoe beïnvloedt de huidige aanpak van monitoring en rapportering de operationele efficiëntie en het risicomangement van SaaS-bedrijven?

Oplossingsgericht:

1. Welke architectuur en technologieën zijn het meest geschikt om een geautomatiseerd securitymonitorsysteem te ontwikkelen dat past binnen een DevOps-omgeving (bijv. Elastic Stack, Wazuh, OpenSearch)?
2. Hoe kan eenvoudige anomaly-detectie op SaaS login-auditlogs, als aanvulling op rule-based detectieregels, bijdragen aan het identificeren van ongebruikelijke authenticatiepogingen?
3. Welke data-integratieprocessen (APIs, agents, pipelines) zijn het meest efficiënt om logs en securitydata uit het geselecteerde SaaS-platform en de DevOps-pipeline te verzamelen en te normaliseren?
4. Hoe kan automatische rapportgeneratie worden geïmplementeerd zodat rapporten zowel technische details als managementinzichten bevatten?
5. Welke prestatie-indicatoren (KPIs) kunnen gebruikt worden om het succes van het systeem te meten, zoals detectiesnelheid, foutposities en respons-tijd?

6. Hoe kan de beveiliging en betrouwbaarheid van het geautomatiseerde systeem zelf worden gegarandeerd binnen de bestaande DevOps-pipeline?

Doelstelling: Het ontwikkelen van een proof-of-concept van een gecentraliseerd systeem dat:

- Securitydata uit het geselecteerde SaaS-platform en de DevOps-pipeline verzamelt via APIs, webhooks en logforwarders;
- Data normaliseert en analyseert om incidenten sneller en accurater te detecteren;
- Automatische rapportages genereert voor technische teams en management;
- KPI's levert om de effectiviteit van monitoring en incident response te meten.

Het concrete eindresultaat is een werkend prototype dat de voordelen van centralisatie, snelle detectie en consistente rapportage aantoont.

A.2. Literatuurstudie

De toenemende nood van bedrijven voor complexe netwerkinfrastructuren en cloud-gebaseerde diensten heeft geleid tot een sterke groei in het aantal beveiligingstools en monitoringoplossingen. Ondanks deze evolutie blijkt het voor veel organisaties moeilijk om een doordachte en effectieve architectuur voor security monitoring te ontwerpen en te implementeren (Obregon, 2015). Dit probleem wordt samengevat door de stelling dat preventie ideaal is, maar detectie noodzakelijk blijft: zonder voldoende detectiemechanismen blijven incidenten onopgemerkt, ongeacht de ingezette preventieve maatregelen (Obregon, 2015).

Een fundamenteel punt binnen security monitoring is zichtbaarheid: men kan niet beschermen wat men niet kan zien. Om deze zichtbaarheid te vergroten, is netwerksegmentatie belangrijk binnen een informatiebeveiligingsstrategie. Door het netwerk op te delen in duidelijk afgebakende zones wordt niet alleen de kans verkleind dat een succesvolle aanval zich kan verspreiden, maar wordt ook de analyse van netwerkverkeer vereenvoudigd en gericht (Obregon, 2015). Netwerksegmentatie vormt daarmee de basis voor een veilige netwerkarchitectuur en ondersteunt zowel detectie als respons.

Een tweede cruciaal punt van security monitoring is logbeheer. Logging en monitoring zijn onmisbaar binnen de functies *Detect* en *Respond* (National Institute of Standards and Technology, 2018). Logs bieden inzicht in wat er zich binnen een

organisatie afspeelt en vormen een belangrijke informatiebron voor probleemoplossing en onderzoek. Logmanagement omvat het volledige proces van genereren, verzamelen, transporteren, opslaan, analyseren en uiteindelijk verwijderen van loggegevens uit diverse bronnen (National Institute of Standards and Technology, 2018). Door deze processen te structureren en te centraliseren kunnen organisaties beter omgaan met grote hoeveelheden data.

Incidentdetectie blijft echter een uitdagend aspect van incident response. Incident handlers worden geconfronteerd met onvolledige, tegenstrijdige en verwarrende signalen die geanalyseerd moeten worden om vast te stellen of er daadwerkelijk sprake is van een beveiligingsincident (Cichonski e.a., 2012). Detectiemechanismen zoals intrusion detection systems genereren vaak false positives, en gebruikersmeldingen zijn niet altijd volledig of correct. Het is noodzakelijk elk signaal kritisch te beoordelen voor conclusies worden getrokken.

Scarfone & Mell beschrijven intrusion detection en prevention systemen als systemen die continu events monitoren en analyseren om mogelijke beveiligingsincidenten te detecteren en relevante informatie te loggen ten behoeve van verdere analyse en incident response (Scarfone & Mell, 2007).

In dit kader spelen SIEM-systemen een centrale rol. SIEM-systemen worden gebruikt om data uit diverse bronnen te verzamelen, te normaliseren en te correleren zodat bedreigingen sneller kunnen worden gedetecteerd en erop kan worden gereageerd (González-Granadillo e.a., 2021). Door logs, alerts en events uit verschillende SaaS- en DevOps-bronnen centraal te aggregeren, kan een SIEM de snelheid en accuratesse van detectie verbeteren en consistente rapportages genereren voor technische teams en management. Bovendien benadrukt de literatuur dat SIEM-systemen, hoewel krachtig, beperkingen hebben zoals complexe configuratie en uitdagingen bij anomaly-detectie, wat de keuze voor een gefaseerde en haalbare implementatie binnen deze bachelorproef ondersteunt (González-Granadillo e.a., 2021).

Het SANS Incident Handler's Handbook van Kral (2011) beschrijft hoe incident response praktisch wordt uitgevoerd, inclusief het verzamelen, analyseren en correleren van security events, het loggen van relevante informatie, en het opvolgen van incidenten (Kral, 2011). Dit raamwerk ondersteunt de opzet van dit onderzoek.

Binnen incident response wordt een onderscheid gemaakt tussen *precursors* en *indicators*. Precursors zijn signalen van mogelijke toekomstige aanvallen, zoals kwetsbaarheidsscans of exploit-aankondigingen. Indicators zijn signalen dat een incident mogelijk al heeft plaatsgevonden, zoals malwaredetecties of mislukte inlogpogingen. Indicatoren vormen de kern van operationele security monitoring (Cichonski e.a., 2012).

Tot slot definieert (Cichonski e.a., 2012) een computer security incident als een daadwerkelijke of dreigende schending van beveiligingsbeleid of gangbare beveiligingspraktijken. Voorbeelden zijn datalekken, malware-infecties, denial-of-service-aanvallen en ongeautoriseerde toegang tot gevoelige gegevens. Een gestructureerde incident response-capaciteit helpt organisaties schade te beperken, dienstverlening te herstellen en lessen te trekken voor toekomstige incidenten.

Prates & Pereira (2024) geven aan dat security in DevOps niet apart kan staan, maar direct ingebouwd moet worden in het ontwikkel- en uitrolproces. Dit ondersteunt de keuze om in dit onderzoek een proof-of-concept te maken dat security monitoring en incident response automatiseert voor één SaaS-platform en één DevOps-pipeline (Prates & Pereira, 2024).

A.3. Methodologie

Het onderzoek wordt uitgevoerd in vier fasen:

1. **Probleemanalyse:** Samenwerking met Devinity.eu om huidige monitoring-tools, workflows en tekortkomingen in kaart te brengen. Analyse van bestaande logs, incidenten en KPI's.
2. **Architectuurontwerp:** Selectie van technologieën zoals Elastic Stack, Wazuh en OpenSearch voor het verzamelen, normaliseren en analyseren van data. Definieren van datastromen via API's, webhooks en logforwarders.
3. **Proof-of-concept ontwikkeling:** Implementatie van centrale logging, correlatie- en analysemethoden, hoofdzakelijk rule-based, aangevuld met optionele en eenvoudige anomaly-detectiemechanismen. Automatische rapportage wordt geïmplementeerd en getest binnen de geselecteerde DevOps-pipeline.
4. **Evaluatie en KPI-meting:** Meten van detectiesnelheid, aantal false positives, responstijd en percentage automatisch afgehandelde incidenten. Analyse van resultaten en aanbevelingen voor optimalisatie.

Voor de evaluatie van het proof-of-concept wordt gebruikgemaakt van gesimuleerde testdata en beperkte, niet-gevoelige logdata uit test- of stagingomgevingen. Deze testdata bestaat uit authenticatielogs, auditlogs en API-events van het geselecteerde SaaS-platform en de DevOps-pipeline, aangevuld met doelbewust gegenereerde security-events zoals mislukte loginpogingen en eenvoudige configuratiefouten. Zo kunnen realistische scenario's worden getest zonder productie-incidenten.

De prestaties van het proof-of-concept worden vergeleken met een baseline waarin dezelfde events enkel via standaard logging- en alertmechanismen beschikbaar

zijn. De vergelijking focust op de mate waarin het PoC events centraliseert, sneller detecteert en duidelijkere rapportering oplevert.

De KPI's worden gemeten met eenvoudig reproduceerbare methoden: detectiesnelheid als tijd tussen event en alert, aantal false positives als percentage van gegenereerde alerts zonder corresponderend test-event, responstijd als tijd tussen alert en initiële actie, en percentage automatisch afgehandelde incidenten als aandeel events met vooraf ingestelde acties.

Deze aanpak bouwt voort op de NIST-aanbevelingen voor intrusion detection en prevention systemen, waarin het vastleggen, analyseren en correleren van security events centraal staat voor effectieve detectie en respons (Scarfone & Mell, 2007). Tegelijkertijd baseert de architectuurkeuze zich op kerncomponenten van SIEM-systemen, zoals gecentraliseerde logverzameling, normalisatie, rule engines en event monitoring (González-Granadillo e.a., 2021). Het proof-of-concept is echter geen volledig commercieel SIEM-systeem. In plaats daarvan betreft het een lichtgewicht, modulair systeem dat de belangrijkste principes van centralisatie en incidentdetectie toepast binnen een specifieke DevOps- en SaaS-context.

Tijdsplanning:

- Fase 1: 3 weken - requirements-analyse en dataverzameling.
- Fase 2: 2 weken - architectuurontwerp en selectie van tools.
- Fase 3: 6 weken - implementatie van proof-of-concept.
- Fase 4: 2 weken - evaluatie, metingen en rapportering.

Deliverables per fase:

- Fase 1: Analyseverslag van huidige situatie en probleemstelling.
- Fase 2: Ontwerpspecificatie van de architectuur en datastromen.
- Fase 3: Werkend proof-of-concept systeem.
- Fase 4: Evaluatierapport met KPI's, conclusies en aanbevelingen.

A.4. Verwacht resultaat, conclusie

Het verwachte resultaat is een proof-of-concept van een gecentraliseerd security-monitoringsysteem dat:

- Securitydata van het geselecteerde SaaS-platform en de DevOps-pipeline centraliseert en normaliseert;

- Incidenten sneller en betrouwbaarder detecteert via correlatie en rule-based analyse;
- Automatische rapportages genereert voor technische teams en management;
- KPI's levert zoals MTDD, MTTR, aantal false positives en responstijd.

Eenvoudige anomaly-detectie wordt optioneel onderzocht, als aanvulling op rule-based detectieregels. Uitgebreide ML-methoden vallen buiten de scope van deze bachelorproef en worden enkel kwalitatief besproken indien toegepast.

De meerwaarde voor de doelgroep ligt in hogere operationele efficiëntie, betere risico-inschatting en snellere incidentafhandeling. Het onderzoek levert praktische inzichten en een concrete demonstratie van een geautomatiseerd monitoring- en incident response-systeem dat haalbaar is binnen de beschikbare tijd en middelen, dankzij de beperkte en realistische scope.

Bibliografie

- Billawa, P., Tukaram, A. B., Ferreyra, N. E. D., Steghöfer, J.-P., Scandariato, R., & Simhandl, G. (2022). SoK: Security of Microservice Applications: A Practitioners' Perspective on Challenges and Best Practices. *Proceedings of the 17th International Conference on Availability, Reliability and Security (ARES 2022)*. <https://doi.org/10.1145/3538969.3538986>
- Cichonski, P., Millar, T., Grance, T., & Scarfone, K. (2012). *Computer Security Incident Handling Guide* (tech. rap. Nr. NIST Special Publication 800-61 Revision 2). National Institute of Standards en Technology. <https://doi.org/10.6028/NIST.SP.800-61r2>
- González-Granadillo, G., González-Zarzosa, S., & Díaz, R. (2021). Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures. *Sensors*, 21(14), 4759. <https://doi.org/10.3390/s21144759>
- Gruhn, V., & Köhntopp, M. (2021). DevOps and Security – Bridging the Gap: A Systematic Literature Review. *Journal of Systems and Software*, 185, 111248. <https://doi.org/10.1016/j.jss.2021.111248>
- Jakku, P. C. (2025). Transforming DevOps: The Critical Role of Monitoring and Logging in Effective Troubleshooting and Optimization. *The New World Foundation*. https://www.researchgate.net/publication/388613227_Transforming_DevOps_The_Critical_Role_of_Monitoring_and_Logging_in_Effective_Troubleshooting_and_Optimization
- Kral, P. (2011). *Incident Handler's Handbook* (tech. rap.). SANS Institute.
- National Institute of Standards and Technology. (2018). *Framework for Improving Critical Infrastructure Cybersecurity* (tech. rap. Nr. Version 1.1). NIST. Gaithersburg, MD.
- Obregon, L. (2015). *Infrastructure Security Architecture for Effective Security Monitoring* (White Paper). SANS Institute. <https://www.sans.org/white-papers/36512>
- Persistence Market Research. (2025). IT Infrastructure Monitoring Market Size, Share, and Growth Forecast, 2025–2032 [Accessed 2026].
- Prates, L., & Pereira, R. (2024). DevSecOps practices and tools. *International Journal of Information Security*, 24(1). <https://doi.org/10.1007/s10207-024-00914-z>
- Scarfone, K. A., & Mell, P. (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)* (tech. rap. Nr. NIST SP 800-94). National Institute of Standards en Technology. <https://doi.org/10.6028/NIST.SP.800-94>

Tuyishime, E., Balan, T. C., Cotfas, P. A., Cotfas, D. T., & Rekeraho, A. (2023). Enhancing Cloud Security—Proactive Threat Monitoring and Detection Using a SIEM-Based Approach. *Applied Sciences*, 13(22), 12359. <https://doi.org/10.3390/app132212359>